

# HelixMemory

## HelixMemory

أربعة محركات رائدة، مُدمجة — AI دماغ ذاكرة واحد لوكلاء

### المُلخص

HelixMemory هو Go SDK (Mem0، Cognee، Letta، Graphiti) يوحد أربعة أنظمة ذاكرة رائدة AI في محرك ذاكرة معرفية واحد يبحث فيها بشكل متوازٍ ويدمج النتائج. يمنح تطبيقات (Graphiti) طبقة ذاكرة واحدة متينة، خالية من التكرار، ومُرتبة مجدداً بدلاً من أربع طبقات غير مترابطة.

### وصف مختصر

HelixMemory هو Go SDK يدمج Mem0، Cognee، Letta و Graphiti في محرك AI، يوجّه عمليات الكتابة بذكاء، يبحث في كل خلفية بشكل متوازٍ. AI ذاكرة معرفية موحد لتطبيقات، ويدمج النتائج عبر خط أنابيب ثلاثي المراحل: جمع، إزالة التكرار، وإعادة الترتيب.

### وصف مفصل

الوحدة (Go SDK) يُقدّم كـ AI هو محرك ذاكرة معرفية موحد لتطبيقات HelixMemory الإصدار 1.25+). يقوم رهبانه التأسيسي على أن أي Go، digital.vasic.helixmemory، مشروع ذاكرة واحد لن يكون الأفضل في كل شيء — لذا بدلاً من إعادة تنفيذ الذاكرة من الصفر وتوريث نقاط الضعف لمشروع واحد، يقوم بتنسيق أربعة أنظمة رائدة ويتيح لكل منها اللعب على نقاط قوته لبناء مخططات المعرفة الدلالية Cognee، لاستخراج الحقائق الديناميكية وإدارة التفضيلات Mem0، لتشغيل وكلاء ذو حالة مع كتل ذاكرة قابلة للتعديل وحوسبة أثناء النوم Letta، ECL عبر مسارات لمخطط معرفة ثنائي الزمن يمكنه التفكير في كيفية تغير الحقائق عبر الزمن Graphiti و.

إن محرك الدمج هو ما يحول تلك المخازن الأربعة المستقلة إلى عقل واحد. في مسار الكتابة، يتم تصنيف كل ذاكرة واردة حسب المحتوى وتوجيهها إلى الخلفية الأنسب لحفظها. في مسار القراءة، يتم نشر الاستعلام عبر جميع الخلفيات بشكل متوازٍ، وتتدفق النتائج الأولية إلى خط أنابيب دمج ثلاثي المراحل الجمع، إزالة التكرار، ثم إعادة الترتيب عبر المصادر — بحيث لا يرى المتصل أربعة مجموعات — نتائج مشوشة ومتداخلة، بل إجابة واحدة نظيفة ومرتبّة. تُغلف قواطع الحوائث كل خلفية لتدهور الأداء بشكل رشيق: عندما يتعطل محرك واحد، ينفث قاطعه وتبقى الخلفيات المتبقية تعمل بدلاً من سحب

القابلة للإسقاط، فإنه يحل محل MemoryStore طبقة الذاكرة بأكملها معها. ولأن المحرك ينفذ واجهة Prometheus مزود ذاكرة بسيط مباشرة — دون الحاجة لإعادة هندسة المتصل — وتعرض مقاييس تفاصيل التوجيه والدمج لتحقيق قابلية المراقبة الكاملة.

AI Helix المجموعة الأوسع من وكلاء HelixAgent كطبقة ذاكرة لـ HelixMemory تم بناء ويحمل معه منهج العائلة في اختبار مكافحة الخداع إلى مجال الذاكرة: يقوم مشغل التحدي المدمج بتشغيل مسارات الكود الإنتاجية الحقيقية — التوجيه، الدمج، المترجم، قاطع الدائرة — بينما يقوم غلاف الطفرات المزدوج بقلب الثوابت عمداً لإثبات أن الاختبارات تفشل بالفعل عندما يكون المنطق معطوباً، بحيث تعني المجموعة الخضراء شيئاً حقيقياً.

## لماذا قمنا ببنائه

### المحتوى

إلى ذاكرة طويلة الأمد وعالية الجودة، لكن النظام البيئي الحالي متشرد — فكل AI يحتاج عملاء يتفوق في جانب ويخفق في جوانب (Mem0، Cognee، Letta، Graphiti) مشروع ذاكرة سطح ذاكرة موحد يجمع نقاط القوة دون إجباره HelixAgent لمنح HelixMemory أخرى. بُني على الارتباط بأي منها بشكل حصري.

## لماذا يُعد هذا تغييراً جذرياً في قواعد اللعبة

إنه ينهي الاختيار القسري. أربعة أنظمة ذاكرة كانت تتنافس عادةً على نفس الفتحة أصبحت الآن، خلفيات تكميلية خلف واجهة واحدة — وبذلك تحصل التطبيقات على استخراج الحقائق الديناميكي مع معالجة، بالتزامن ورسومات المعرفة الدلالية، وذاكرة العملاء ذات الحالة، والاستدلال ثنائي الزمن التكرارات وإعادة الترتيب عبر المصادر تلقائياً. ما لم يكن ممكناً من قبل هو اعتبار سؤال “أي محرك الاستفادة من جميع نقاط القوة معاً خلف واجهة HelixMemory ذاكرة نتبنى؟” معضلة زائفة: يتيح دون وراثه نقاط الضعف لأي محرك أو الائتلاف بحصرية واحدة، MemoryStore واحدة قابلة للتضمين

## ما الجديد في الابتكار

- الذي يعيد (جمع → إزالة التكرارات → إعادة ترتيب عبر المصادر) الدمج متعدد الخلفيات مجموعة نتائج مرتبة واحدة، بدلاً من ربط المتصل بمخزن واحد.
- الذي يصنف كل ذاكرة حسب المحتوى ويرسلها إلى المحرك الأنسب التوجيه الذكي للكتابة. لحفظها، بحيث تصل البيانات الصحيحة إلى المخزن المناسب.
- عبر قواطع دوائر لكل خلفية — فشل المحرك يُعزل ولا يكون مميتاً، ويستمر الباقي التدهور الرشيق في الخدمة.
- التي تعيد تنظيم الذاكرة خلال فترات (Letta عبر) الحوسبة الموحدة أثناء فترات السكون. الخمول بدلاً من الاعتماد على وقت الاستعلام فقط.
- مشغل تحديات يعمل على كود الإنتاج الفعلي، مقترناً بمغلف تحويل: التحقق من عدم التلاعب. يجب أن يفشل عند قلب الثابت — مما يثبت أن بوابة الاختبار ليست مجرد تحسين حاصل.

## أكبر التحديات التقنية وكيف تم التغلب عليها

- كل محرك يعيد الذاكرة — مواءمة أربعة خلفيات غير متجانسة في مجموعة نتائج متماسكة بشكل مختلف، ودمجها بشكل سطحي ينتج عنه تكرارات وترتيبات غير قابلة للمقارنة. تم حل

المشكلة بمحرك دمج مُنظَّم يجمع البيانات من المصادر المختلفة، يزيل التكرارات، ويعيد ترتيبها على أساس مشترك، مع تأكيد الثابت المدمج في الاختبارات لضمان عدم فقدان النتائج أو احتسابها مرتين بشكل صامت.

- يجب ألا يتوقف النظام بأكمله بسبب محرك ذاكرة — **البقاء قيد التشغيل عند تعطل خلفية ما** يتعذر الوصول إليه. تم حل المشكلة بقواطع دوائر لكل خلفية تتبع آلية حالة مغلقة → مفتوحة (بعد تجاوز عتبة الفشل) → نصف مفتوحة (بعد انتهاء المهلة)، لعزل الخلفية المعطلة والاستمرار في الخدمة من الخلفيات السليمة حتى تعافياها.
- مجموعة اختبارات خضراء لا معنى لها إذا لم — **إثبات فعالية منطق الذاكرة وليس مجرد تجميعها** تكن الاختبارات قادرة على الفشل. تم حل المشكلة بمشغل تحديات داخلي يشغل كود الإنتاج الفعلي (التوجيه، الدمج، المترجم، قاطع الدائرة) ومغلف تحويل مزدوج يقلب الثوابت ويتطلب فشل الاختبارات، لضمان أن البوابة ليست تحصيلاً حاصلًا.

المحتوى

## مجموعة الأدوات التقنية

- **Go** — (الإصدار 1.25 فما فوق) وبينيتها التنفيذية؛ اختبرت لأن توزيع القراءة SDK نواة هذه Go المتوازية عبر أربعة أنظمة خلفية يمثل مشكلة توافقية، وتجعل الـ "غوروتينات" في (MemoryStore) العملية غير مكلفة، بينما توفر أنواع الواجهات فيها نقطة وصل نظيفة واحدة يمكن للمستدعين الاعتماد عليها.
- **Mem0** — النظام الخلفي لاستخراج الحقائق الديناميكية وإدارة التفضيلات؛ يُستخدم لجزء "الذاكرة المتعلقة بـ" ما الذي يفضل المستخدم حقًا / وما هي الحقائق التي ظهرت.
- **Cognee** — ECL: النظام الخلفي للرسم البياني الدلالي للمعرفة، مبني على مسارات؛ يُستخدم لتخزين المعرفة المهيكلية المترابطة بدلاً من الحقائق المسطحة.
- **Letta** — النظام الخلفي لوضعية الوكلاء التنفيذية مع كتل ذاكرة قابلة للتعديل وحوسبة أثناء فترات السكون؛ يُستخدم حيث يجب أن تستمر الذاكرة كحالة حية للوكلاء ويتم دمجها خلال فترات الخمول.
- **Graphiti** — النظام الخلفي للرسم البياني ثنائي الزمن للمعرفة؛ يُستخدم للاستدلال حول كيفية تغير الحقائق والعلاقات عبر الزمن، وليس فقط قيمتها الحالية.
- **PostgreSQL + Neo4j + Redis** — مخازن البيانات الحقيقية التي تعمل عليها — **make infra-start** بحيث تختبر الأنظمة الخلفية، ونُشأ لاختبار التكامل الفعلي عبر المجموعة البنية التحتية الحية بدلاً من النماذج الوهمية.
- **Prometheus** — المقاييس والرصد المدمجان عبر مسار الدمج، بحيث يمكن قياس سلوك التوجيه والدمج في الإنتاج بدلاً من أن يكون صندوقاً أسود.
- يُحافظ عليه لتمكين أي طبقة (helixmemory\_) سطح نصوص مُسمى — **واجهة المترجم الدولي** مستقبلية موجهة للمستخدم من التعريب دون الحاجة لإعادة هيكلة النواة.

## ملاحظات الحالة والصريح

- **HelixAgent** يعمل؛ بُني كطبقة ذاكرة لـ SDK. الحالة: تجريبية.
- غير مُتحقق منها / لم — **GitHub API** لم يُكشف عن أي رخصة عبر. الرخصة: لم تُحدد بعد تُعلن.
- الأرقام الدقيقة المذكورة في memory. يشير إلى المستودع "HelixMemory" الاسم المعروف هي ادعاءات مقدمي الخدمات الخارجية، وليست قياسات README ملف HelixMemory، وقد تم حذفها من هنا.

أساسية-Helix: درجة الأولوية